

Proyecto Final PGII

Camila Gonzalez Restrepo, Isabella
Hincapié Bocanegra, Luisa Fernanda
Hernández Ramírez



UNIVERSIDAD
DEL QUINDÍO
Res. MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN



UNIQUINDÍO
en conexión territorial

www.uniquindio.edu.co

Pensamiento Computacional

1. Abstracción

¿Qué se solicita finalmente?

Se pide desarrollar un sistema orientado a objetos en Java con interfaz JavaFX que permita a dos tipos de usuarios (cliente y administrador) gestionar todo el ciclo de vida de un evento: desde su creación hasta la venta, pago, cancelación y generación de reportes. El sistema debe manejar recintos físicos con zonas y asientos numerados, y aplicar patrones de diseño de forma explícita y justificada.

¿Qué información es relevante?

El sistema gira en torno a nueve entidades principales con atributos y estados propios:

Entidad	Descripción y atributos clave
Usuario	idUsuario, nombre completo, correo electrónico, teléfono, métodos de pago simulados, rol (cliente o administrador).
Evento	idEvento, nombre, categoría, ciudad, fecha/hora, estado (Borrador, Publicado, Pausado, Cancelado, Finalizado) y políticas de cancelación/reembolso.
Recinto	idRecinto, nombre, dirección, ciudad. Contiene un conjunto de zonas.
Zona	idZona, nombre (VIP, Preferencial, General), capacidad, precio base y configuración de asientos.
Asiento	idAsiento, fila, número, estado (Disponible, Reservado, Vendido, Bloqueado).
Compra	idCompra, usuario asociado, evento asociado, fecha de creación, total, estado (Creada, Pagada, Confirmada, Cancelada, Reembolsada, Incidencia).
Entrada	idEntrada, zona, asiento (si aplica), precio final calculado, estado (Activa, Usada, Anulada).
Pago	idPago, método de pago, monto y referencia de la compra asociada.
Incidencia	idIncidencia, tipo, descripción, fecha y entidad afectada (evento, compra o usuario).



Tarifa	precioBase, porcentajeImpuesto, comisionServicio
ServicioAdicional	Acceso VIP, seguro de cancelación, merchandising, parqueadero. Cada uno con nombre y costo propio.

¿Cómo se puede agrupar la información?

Los datos se organizan en tres grandes agrupaciones conceptuales:

- **Dimensión del lugar:** Recinto contiene Zona, y Zona contiene Asiento. Esta jerarquía es de composición porque los asientos no tienen sentido sin su zona, ni las zonas sin su recinto.
- **Dimensión del evento:** Evento está asociado a un Recinto y a sus Zonas. El evento define cuáles zonas y asientos están disponibles para venta.
- **Dimensión de la transacción:** Usuario realiza Compra, la cual contiene Entrada y opcionalmente ServicioAdicional. Al confirmarse el pago, se genera un Pago y se actualizan los estados de los asientos involucrados.

2. Descomposición

¿Qué funcionalidades se solicitan?

Las funcionalidades se dividen en cuatro grupos según su responsabilidad:

Gestión de catálogo (solo administrador)

- Crear, actualizar, publicar, pausar y cancelar eventos.
- Administrar recintos, zonas y asientos.
- Gestionar usuarios (crear, actualizar, eliminar, listar).

Navegación y selección (usuario)

- Explorar eventos disponibles con filtros por fecha, ciudad, categoría y precio.
- Consultar detalle de un evento con disponibilidad por zona y asiento.
- Seleccionar entradas y visualizar mapa de asientos.

Proceso de compra (usuario y sistema)

- Crear compra, agregar servicios adicionales y procesar pago.
- Generar entradas y actualizar estado de asientos.

- Enviar notificaciones de cambio de estado al usuario.

Gestión post-compra

- Consultar historial de compras con filtros.
- Solicitar cancelaciones según políticas definidas.
- Descargar comprobantes y reportes en CSV o PDF.
- Registrar incidencias operativas.

Panel administrativo

- Ver métricas: ventas por periodo, ocupación por zona, ingresos por servicios adicionales, tasa de cancelación y top eventos.
- Visualizar métricas con JavaFX Charts (líneas, barras, pie).

¿Cómo se distribuyen las funcionalidades entre las clases?

Cada clase del dominio tiene responsabilidades bien delimitadas, siguiendo el principio SRP:

- **Usuario:** administra sus propios datos, métodos de pago y puede consultar sus compras, pero no ejecuta la lógica de compra directamente.
- **GestorDeCompras (servicio):** orquesta el flujo de compra: valida disponibilidad, reserva asientos, crea la compra, delega el pago y genera las entradas.
- **GestorDeEventos:** maneja el ciclo de vida de los eventos y coordina cambios de estado que afectan a múltiples compras.
- **CalculadorDeTarifa:** aplica la lógica de precio según zona, servicios adicionales y posibles descuentos. Aquí aplica el patrón Strategy.
- **GeneradorDeReportes:** se encarga exclusivamente de exportar a CSV o PDF, sin mezclar esa lógica con las entidades del dominio.
- **GestorDeNotificaciones:** observa cambios de estado en eventos y compras para avisar a los usuarios afectados (patrón Observer).

3. Reconocimiento de Patrones

Patrones creacionales

1. Singleton

La conexión a la base de datos o el catálogo de eventos cargado en memoria debe existir como una única instancia compartida. Sin este patrón, cada módulo podría



crear su propia copia y los datos quedarían desincronizados. Se aplicaría a clases como GestorDeEventos o el repositorio central de datos.

2. Factory Method

EventoFactory crea el objeto concreto (Concierto, Conferencia, Teatro) según el CategoriaEvento recibido, sin que el código cliente necesite conocer la clase concreta.

3. Builder

Construir una compra es un proceso en múltiples pasos: selección de zona, asientos, servicios y método de pago. El patrón Builder permite ir ensamblando la compra de forma controlada antes de confirmarla, evitando constructores con muchos parámetros y estados inválidos intermedios.

Patrones estructurales

1. Decorator

Agregar servicios adicionales a una compra funciona como capas que envuelven el precio base. ServicioVIP, ServicioSeguro, ServicioParqueadero y los demás envuelven una ICompra y suman su costo al total. Cada decorador añade responsabilidad sin modificar la compra original.

2. Facade

CompraFacade expone un único punto de entrada que coordina internamente CompraService, EntradaService, PagoService, NotificacionService e IncidenciaService, simplificando la interacción del cliente con todos esos subsistemas.

3. Adapter

CsvReporteAdapter y CdfReporteAdapter adaptan librerías externas (Apache POI y Apache PDFBox) a la interfaz IGeneradorReporte que maneja el sistema. El resto del código solo conoce exportar() — no sabe ni le importa qué librería hay por debajo.

Patrones de comportamiento

1. Strategy

El procesamiento del pago puede variar: tarjeta de crédito, débito o PSE. PagoService usa una PagoStrategy intercambiable en tiempo de ejecución sin modificar la clase que procesa el pago.

2. Observer

Cuando un evento cambia su estado (por ejemplo, de Publicado a Cancelado), todos los usuarios con compras activas deben ser notificados. El evento actúa como sujeto observable y los usuarios (o el gestor de notificaciones) como observadores. Esto desacopla el emisor del cambio de los receptores de la notificación.

3. State

Una Compra se comporta distinto según en qué etapa esté. CompraCreada, CompraPagada, CompraConfirmada, CompraCancelada y CompraReembolsada encapsulan cada comportamiento y controlan qué transiciones son válidas en cada momento.

4. Diseño de la Solución (Codificación Conceptual)

El proyecto se organizaría en capas bien separadas:

- **Modelo:** las entidades del dominio (Usuario, Evento, Compra, Recinto, Zona, Asiento, Entrada, Pago, Incidencia, ServicioAdiciona, Tarifal).
- **Servicio:** la lógica de negocio (GestorDeCompras, GestorDeEventos, GeneradorDeReportes, GestorDeNotificaciones).
- **Repositorio:** gestión de datos en memoria o archivos; un repositorio por entidad principal.
- **Patrón:** las clases que implementan los patrones de diseño (estrategias, decoradores, builders, proxy, fachada, factory).
- **UI:** las pantallas JavaFX para usuario y administrador.
- **Util:** herramientas transversales como exportadores CSV/PDF y validadores.

La codificación será implementada luego de la aprobación de este informe.

Patrones Creacionales

Estos patrones tienen que ver con cómo se crean los objetos dentro del sistema. La idea es que la creación no quede regada por todo el código sino que esté controlada y organizada.

1. Singleton

Qué es: Garantiza que ciertas partes del sistema existan una sola vez durante toda la ejecución del programa.

Requisitos que resuelve: RF-003, RF-004, RF-005 Todos los componentes consultan el mismo GestorEventos para explorar, filtrar y verificar disponibilidad de asientos.

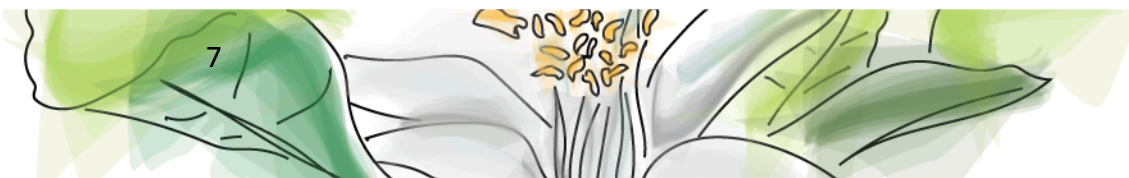
El problema: GestorEventos es la clase que centraliza todos los eventos del sistema. Si cualquier parte del código pudiera crear su propia instancia, habría múltiples gestores con listas de eventos distintas y desincronizadas; uno tendría unos eventos, otro tendría otros, y el sistema perdería consistencia.

Por qué este y no otro: Con Singleton se garantiza que toda la aplicación comparte exactamente el mismo gestor con exactamente la misma lista de eventos. Se descartó una clase con métodos estáticos porque no permitiría implementar interfaces. Se descartó dejar la instanciación libre porque el problema central es precisamente que solo debe existir una.

2. Factory Method

Qué es: Define una forma estándar de crear objetos sin que el resto del código tenga que saber exactamente qué tipo de objeto se va a crear.

Requisitos que resuelve: RF-003, RF-004 Cuando el usuario explora o consulta un evento, el sistema necesita crear el objeto correcto según su categoría (concierto, conferencia, obra de teatro).



El problema: El sistema maneja tres tipos de evento: conciertos, conferencias y obras de teatro. Cada uno tiene atributos distintos — un concierto tiene artista y género musical, una conferencia tiene ponente y temática, una obra tiene director y género. Si cada parte del código que necesita crear un evento instancia directamente la clase concreta con `new Concierto()` o `new ObraTeatro()`, el sistema queda atado a esas clases y agregar un nuevo tipo de evento obligaría a rastrear y modificar todos esos puntos.

Por qué este y no otro: Con Factory Method, la decisión de qué clase instanciar queda en un solo lugar. Se descartó Abstract Factory porque no hay familias de objetos relacionados que crear juntos, solo un único producto que varía según la categoría. Se descartó el constructor directo porque acopla al llamador con las clases concretas.

3. Builder

Qué es: Permite construir un objeto complejo paso a paso, asegurando que esté completo antes de usarlo.

Requisitos que resuelve: RF-006, RF-007 Construir una compra paso a paso (zona, asientos, servicios, método de pago) antes de confirmarla y procesarla.

El problema: Una compra no se crea de una vez: el usuario primero elige la zona, luego los asientos, después agrega servicios opcionales y finalmente confirma. Si se va armando el objeto compra durante todo ese proceso, hay momentos en que está a medias y podría usarse antes de estar lista, causando errores.

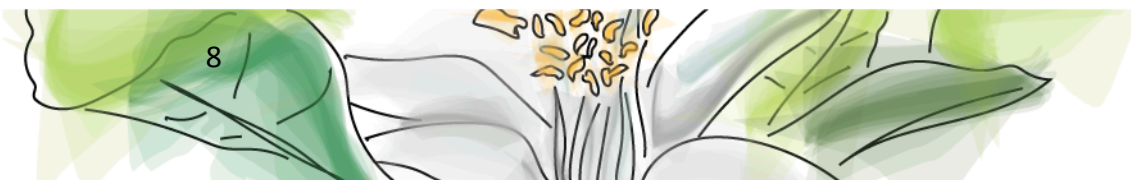
Por qué este y no otro: El Builder va acumulando todas las decisiones del usuario en un objeto intermedio y solo crea la compra real cuando todo está listo. Se eligió sobre un constructor con muchos parámetros porque en Java eso se vuelve ilegible rápidamente, y sobre Factory Method porque aquí el problema es el ensamblaje en pasos, no la elección de tipo.

Patrones Estructurales

Estos patrones organizan cómo se relacionan y combinan las clases entre sí. Ayudan a que el sistema sea más fácil de extender sin tener que reescribir lo que ya funciona.

1. Decorator

Qué es: Permite agregar funcionalidades nuevas a un objeto envolviéndolo en capas, sin modificar la clase original.



Requisitos que resuelve: RF-009 Cada servicio adicional (VIP, seguro, merchandising, parqueadero, acceso preferencial) se agrega como una capa sobre la compra base.

El problema: Un usuario puede comprar una entrada y agregarle extras: seguro, parqueadero, acceso VIP, merchandising. El problema es que las combinaciones son muchas y no se pueden predecir; alguien puede querer solo el seguro, otro el VIP con parqueadero, otro los cuatro juntos. Crear una subclase para cada combinación posible sería inmanejable.

Por qué este y no otro: Con Decorator, cada servicio es una capa que envuelve la compra y suma su costo al total. Se pueden combinar libremente en cualquier orden sin que ninguno sepa qué otros hay encima o debajo. Se descartó la herencia porque una subclase por cada combinación posible haría el proyecto inmanejable, y Strategy porque el problema no es elegir una forma de hacer algo sino acumular extras sobre un mismo objeto.

2. Adapter

Qué es: Permite que clases con interfaces incompatibles puedan trabajar juntas. En este caso, adapta diferentes formas de generar reportes para que todas se puedan usar mediante una misma interfaz.

Requisitos que resuelve: RF-011 Al descargar reportes, CsvReporteAdapter y PdfReporteAdapter adaptan las librerías externas para generar el archivo en el formato que el usuario eligió.

El problema: El sistema necesita exportar reportes en diferentes formatos, como CSV y PDF. Sin embargo, cada tecnología o librería puede trabajar de forma distinta, por ejemplo Workbook para Excel/CSV o PDDocument para PDF. Si la clase Reporte tuviera que manejar directamente cada librería, quedaría muy acoplada y sería más difícil agregar nuevos formatos después.

Por qué este y no otro: El Adapter permite adaptar diferentes formas de exportación para que todas puedan ser utilizadas de manera uniforme mediante una misma interfaz. Así, el sistema no necesita conocer los detalles internos de cada formato, solo llama una operación común. Se descartó Facade (en este problema) porque no se busca simplificar un proceso compuesto por varios subsistemas, sino hacer compatibles componentes que tienen formas distintas de trabajar.

3. Facade

Qué es: Ofrece una interfaz simple que esconde la complejidad de varios subsistemas que trabajan juntos.

Requisitos que resuelve: RF-006, RF-007 El proceso de compra coordina internamente varios servicios. El usuario sólo ejecuta una acción, CompraFacade orquesta todo por detrás.

El problema: El proceso de compra involucra verificar disponibilidad de asientos, crear la compra, procesar el pago, generar las entradas, enviar notificaciones y registrar incidencias si algo falla. Si la pantalla de compra tiene que coordinar directamente con todos esos módulos, queda muy acoplada a ellos. Cualquier cambio en cómo funciona el pago, por ejemplo, obliga a tocar también la pantalla.

Por qué este y no otro: La Facade expone una interfaz simple mediante una clase fachada que orquesta todo internamente. La pantalla solo habla con ese método y no sabe nada de lo que pasa adentro. Se descartó el Mediator porque los subsistemas no necesitan comunicarse entre sí, sólo necesitan ser llamados en secuencia por alguien que los coordine.

Patrones de Comportamiento

Estos patrones definen cómo se comunican y colaboran los objetos cuando el sistema está en funcionamiento. Ayudan a que los cambios en una parte no rompan las demás.

1. Strategy

Qué es: Permite tener varias formas de hacer lo mismo e intercambiarlas fácilmente según el contexto.

Requisitos que resuelve: RF-007 Al momento de pagar, el usuario elige su método (tarjeta crédito, débito, PSE) y el sistema usa la estrategia correspondiente sin cambiar la lógica de pago.

El problema: El precio de una entrada no siempre se calcula igual. Puede ser precio fijo, con descuento por temporada, precio especial para grupos, etc. Si se ponen todas esas reglas dentro de la clase que maneja la compra con una cadena de condiciones, esa clase crece sin control y cada vez que aparece una nueva regla de precio hay que modificarla.

Por qué este y no otro: Se descartó Template Method porque las tres formas de pago no comparten pasos en común, pues cada una tiene su propio flujo completamente independiente. Strategy permite agregar o quitar métodos de pago sin tocar PagoService.

2. Observer

Qué es: Permite que cuando algo cambia, todos los interesados en ese cambio se enteren automáticamente sin que el que cambió tenga que avisarle a cada uno por separado.

Requisitos que resuelve: RF-008 Cuando el estado de un evento cambia, los usuarios con compras activas son notificados automáticamente.

El problema: Cuando el administrador cancela o pausa un evento, todos los usuarios que compraron entradas para ese evento necesitan saberlo. El problema es que el número de usuarios afectados varía en cada caso y no se puede saber de antemano. Si el gestor de eventos tiene que llamar uno a uno a cada usuario para notificarlo, queda atado a saber quiénes son y cómo notificarlos.

Por qué este y no otro: Con Observer, los usuarios se suscriben al evento como observadores. Cuando el estado del evento cambia, él simplemente avisa a todos los suscritos sin conocer sus detalles. Se descartó el Mediator porque la comunicación aquí es en una sola dirección: el evento avisa, los usuarios reciben.

3. State

Qué es: Permite a un objeto alterar su comportamiento cuando cambia su estado interno, haciendo que parezca que el objeto cambia su clase.

Requisitos que resuelve: RF-006, RF-008 Cada estado de la compra (Creada, Pagada, Confirmada, Cancelada, Reembolsada) controla qué acciones son válidas en ese momento.

El problema: Una compra pasa por varias etapas: se crea, se paga, se confirma, se puede cancelar y hasta reembolsar. El problema es que no todas las acciones tienen sentido en cualquier momento — no debería poder confirmarse si todavía no se pagó, ni reembolsarse si no está cancelada. Sin este patrón, esa lógica terminaría siendo una cadena interminable de if (estado == CREADA) dentro de Compra, y agregar un nuevo estado significaría revisar toda esa cadena.

Por qué este y no otro: Se descartó Strategy porque aquí el comportamiento no es intercambiable libremente; cada estado tiene transiciones válidas e inválidas bien definidas, y el propio objeto controla a cuál estado puede pasar. Strategy no modela esa lógica de transición.



UNIVERSIDAD
DEL QUINDÍO
Res. MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN



Ingeniería de Sistemas y Computación

**Tel: (57) 6 735 9300 Ext
Carrera 15 Calle 12 Norte
Armenia, Quindío – Colombia
clondonoidarraga@uniquindio.edu.co**

UNIQUEINDÍO, en conexión territorial

Carrera 15 Calle 12 Norte Tel: (606) 7 35 93 00 Armenia - Quindío - Colombia